Algoritmi Paraleli și Distribuiți
Test Curs (Varianta Exemplu 1)
6 Decembrie 2024, ora 12:00
Timp de lucru: 45 de minute

Numele: _____

Grupa: _____

Reguli de desfășurare a testului:

- Tipul testului este **closed book**. Pe durata acestuia nu se permit consultarea materialelor de studiu, utilizarea dispozitivelor electronice sau colaborarea cu alte persoane.
- Testul constă conține o singură cerință, pentru întreg punctajul. Nu se acordă puncte din oficiu.
- Foaia cu subiectul se vor preda obligatoriu odată cu rezolvarea.

Problemă

Scrieți pseudocodul pentru o colecție de N fire de execuție care construiesc împreună un arbore binar (obișnuit, **nu** de căutare) format din elemente de tipul TreeNode.

Fiecare dintre firele de execuție va primi o listă (vector) cu noduri pe care va trebui să le adauge în arbore, dată sub forma unor perechi de forma (id, părinte) care înseamnă că nodul id trebuie adăugat ca fiu al nodului cu id-ul părinte.

Se garantează că listele de perechi sunt împărțite și ordonate astfel încât firele de execuție să nu intre în deadlock datorită așteptării creării unui alt nod de către un alt fir de execuție. De asemenea, toate thread-urile cunosc: **N** numărul de thread-uri; **M** numărul total de noduri din arbore; **root** - nodul rădăcină (creat inițial cu id = 0 și niciun fiu).

La final, unul dintre thread-uri (ales de voi) va afișa arborele creat, folosind funcția **print_inorder(root)**.

Rezolvați problema folosind Semafoare. Folosirea unei bariere sau blocarea tuturor thread-urilor (inclusiv a celor care au terminat de creat nodurile ce le sunt atribuite) până când toate termină de creat, va echivala pierderii a 25% din punctajul problemei.

Soluția trebuie să maximizeze concurența firelor de execuție. O soluție care serializează operații care nu ar genera condiții de cursă va primi punctaj 0, iar fiecare utilizare a unui busy-waiting se va depuncta cu 25%.

Puteți presupune că există deja implementată funcția **TreeNode *find(ThreeNode *root, int needle)** care caută nodul cu id-ul needle în arborele cu rădăcina root și vă întoarce: nod-ul dacă acesta există; NULL dacă nodul nu a fost găsit. La fel, puteți considera deja implementată funcția **print_inorder**.

Enumerați problemele de sincronizare care ar putea apărea înainte de a scrie codul care le tratează!

Pentru reprezentarea arborelui și a input-ului fiecărui fir de execuție se vor folosi structurile din listing-urile de mai jos.

```
struct TreeNode { // constructor TreeNode(id)
  struct TreeNode *left = NULL;
  struct TreeNode *right = NULL;
  int id = id;
  // Eventual alte elemente adaugate de voi.
};
```

Listing 1: Structura TreeNode.

```
struct Pereche {
  int id;
  int parinte;
};
```

Listing 2: Structura Pereche.

Posibilă soluție

Probleme de sincronizare

- Pe parcursul creării arborelui:
  - Adăugarea unui fiu pentru un nod care nu a fost încă creat;
  - Adăugarea simultană a 2 fii pentru același nod;
- După ce arborele a fost creat:
  - Thread-ul care face afișarea arborelui trebuie să aștepte ca acesta să fie creat complet, înainte de a face afișarea.

Descrierea soluției[1]

- Pentru rezolvarea primei probleme de sincronizare vom considera un vector (*creat*) de M semafoare (câte unul pentru fiecare nod din arbore), toate inițializate cu 0.
  - Înainte de a crea un nod *id*, thread-ul responsabil va aștepta semaforul *creat*[*id*];
  - Atunci când creează nodul *id*, thread-ul responsabil va incrementa cu 2 semaforul *creat*[*id*].
- Pentru a doua problemă de sincronizare, vom adăuga un mutex (m) în fiecare nod. Acesta va fi luat înainte de a adăuga un fiu pentru nodul respectiv și eliberat după, asigurând faptul că existența unui fiu (la stânga/dreapta) nu este testată simultan de 2 thread-uri.
- Pentru a treia problemă vom un contor (*count*) și vom debloca un semafor (*done*) abia atunci când *count* ajunge la N. Accesul la acest contor va fi protejat de un semafor (*count_mutex*) folosit ca mutex. Thread-ul de afișare va aștepta semaforul *done* înainte de a afișa arborele.

Soluție

```
// Variabile partajate
Sem creat[M] = {Semafor(0)}; // M semafoare 0
Sem done(0), count_mutex(1);
int count = 0;
proces Tread[tid = 1 to N] {
    for(Pereche p in lista_mea) {
        ThreeNode nod_nou = ThreeNode(p.id);
        nod_nou.m = Sem(1); // initializez mutex-ul adaugat in nod
       // Astept sa fie creat parintele
       P(creat[p.parinte]);
       TreeNode parinte = find(root, p.parinte);
       // Iau mutex pentru nodul parinte
       P(parinte.m);
       if(parinte.left == NULL) {
              parinte.left = nod_nou
       } else {
              parinte.right = nod_nou;
       }
       // Eliberez mutex pentru parinte
       V(parinte.m);
       // Incrementez semaforul creat[p.id] de doua ori ca sa permit crearea a (maxim) doi fii.
       V(creat[p.id]);
       V(creat[p.id]);
    }
    // La final incremented count si verific daca sunt ultimul
    P(count_mutex);
    count++;
    if (count == N) {
        // Ultimul anunta ca se poate face printarea
        V(done);
    }
    V(count_mutex);
    if(tid == 0) { // Tread-ul 0 printeaza, dupa ce asteapta done
        P(done);
        print_inorder(root);
    }
}
Observatie:
Am presupus un membru in plus (m de tip semafor) pentru fiecare TreeNode. Acesta trebuie sa fie
     initializat cu 1 pentru toate nodurile, inclusiv pt. radacina.
```

Listing 3: Soluție.

[1] Aceasta secțiune poate să nu apară în rezolvarea de la test. Ea apare aici ca să vă explice soluția. În lipsa ei, punctajele parțiale (ce vor fi oferite dacă sunt mici greșeli în pseudocod) vor fi condiționate de a avea destule comentarii în cod pentru a putea înțelege ce ați încercat să faceți.